Université du Luxembourg

Introduction to Message Passing Interface (MPI)

Lesson 8 - MPI Collective Operations 2

Matteo Barborini

University of Luxembourg - HPC Platform

matteo.barborini@uni.lu

4th of November 2026



Université du Luxembourg

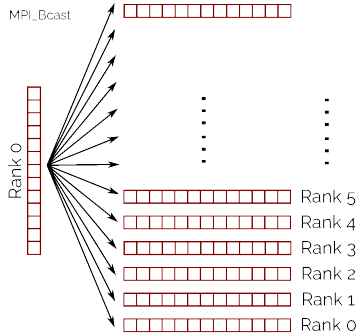
MPI scattering and gathering operations

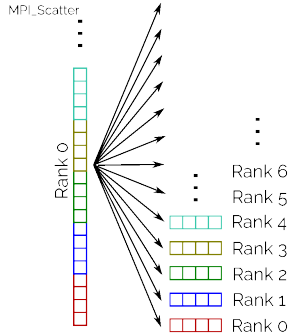


Difference between MPI_Bcast and MPI_Scatter

MPI scattering and gathering operations

To distribute partial data to all processes in a communicator, we can use either partial MPI_Bcast or MPI_Scatter.





MPI_Scatter

MPI scattering and gathering operations

```
int MPI_Scatter  [ MPI_Iscatter ] (
            [in] const void *data_to_scatter,
            [in] int size_of_data_to_scatter,
            [in] MPI_Datatype data_type_to_scatter,
            [out] void *receiving_variable,
            [in] int size_of_receiving_variable,
            [in] MPI_Datatype data_type_to_scatter,
            [in] int rank_that_scatters,
            [in] MPI_Comm comm,
            [out] [MPI_Request *request ] )
```

- Since it is a global operation, every process in the communicator (comm) must call MPI_Scatter (or MPI_Iscatter) with matching parameters (`data_type` for send and receive buffers.);
- The send buffer is only significant for the process with rank `rank_that_scatters` (the root process);
- The send buffer is divided into equal-sized blocks, one for each process in the communicator. The size of each block is determined by `size_of_receiving_variable` and `data_type_to_receive`.
- The size of the receive buffer is the same for all processes.
- MPI_Iscatter as all instantaneous operations, returns immediately and the operation may not have completed when the function returns. The user must ensure that the operation is completed before using the data in the receive buffer. This can be done by calling MPI_Wait or MPI_Test on the request object returned by MPI_Iscatter.

Example of MPI_Scatter

MPI scattering and gathering operations

In the example `8.1.MPI_Scatter.c` we can find a simple implementation of the MPI_Scatter function.

```
// Initialization of global vector
if (mpi_rank == 0)
{
   global_vector = allocate_1d_double(&num_global_elements);
   init_random_vector(10, &num_global_elements, global_vector);
}
// Allocation of local vector initialized to 0
local_vector = allocate_1d_double(&num_local_elements_per_rank);

// Scatter parts of the global vector
MPI_Scatter(&global_vector[0],
            num_local_elements_per_rank,
            MPI_DOUBLE,
            &local_vector[0],
            num_local_elements_per_rank,
            MPI_DOUBLE,
            0,
            MPI_COMM_WORLD);
```

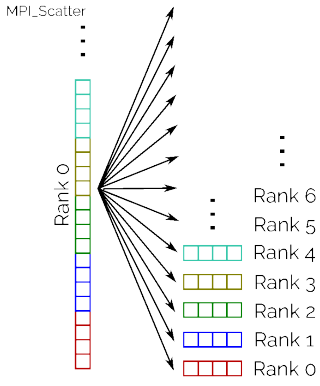
The root process initializes a global vector with random values and scatters it to all processes in the communicator.

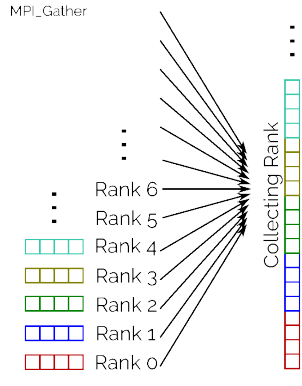
Each process receives a portion of the global vector and stores it in its local vector.

MPI_Scatter and MPI_Gather

MPI scattering and gathering operations

The gather operation is the inverse of the scatter operation. It collects data from all processes in the communicator and gathers it into a single buffer on the root process.





MPI_Gather

MPI scattering and gathering operations

```
int MPI_Gather [MPI_Igather] (
            [in] const void *local_data_to_gather,
            [in] int size_of_local_data_to_gather,
            [in] MPI_Datatype data_type_to_gather,
            [out] void *receiving_variable,
            [in] int size_of_receiving_variable,
            [in] MPI_Datatype data_type_to_gather,
            [in] int rank_that_gathers,
            [in] MPI_Comm comm,
            [out] [ MPI_Request *request ] )
```

- Since it is a global operation, every process in the communicator (comm) must call MPI_Gather (or MPI_Igather) with matching parameters (`data_type` for send and receive buffers.);
- The receive buffer is only significant for the process with rank `rank_that_gathers` (the root process);
- The receive buffer is divided into equal-sized blocks, one for each process in the communicator. The size of each block is determined by `size_of_local_data_to_gather` and `data_type_to_gather`;
- MPI_Igather as all instantaneous operations, returns immediately and the operation may not have completed when the function returns. The user must ensure that the operation is completed before using the data in the receive buffer by calling MPI_Wait or MPI_Test on the request object returned by MPI_Igather.

Example of MPI_Gather

MPI scattering and gathering operations

In the example `8.2.MPI_Gather.c` we can find a simple implementation of the MPI_Gather function.

```
// Initialization of local vector
local_vector = allocate_1d_double(&num_local_elements_per_rank);
init_random_vector(mpi_rank, &num_local_elements_per_rank, local_vector);

// Initialization of global vector
if (mpi_rank == 0)
{
   global_vector = allocate_1d_double(&num_global_elements);
}
// Gather operation to collect local vectors into the global vector
MPI_Gather(&local_vector[0],
          num_local_elements_per_rank,
          MPI_DOUBLE,
          &global_vector[0],
          num_local_elements_per_rank,
          MPI_DOUBLE,
          0,
          MPI_COMM_WORLD);
```

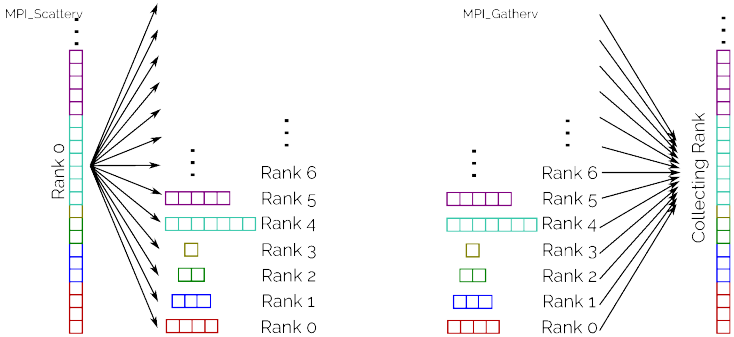
Each local vector is initialized with a different random seed based on the rank of the process.

The root process gathers all local vectors into a global vector.

MPI_Scatterv and MPI_Gatherv

MPI scattering and gathering operations

If we want to scatter or gather data of different sizes, we can use the MPI_Scatterv and MPI_Gatherv functions. These functions allow us to specify the size of the data to be sent or received for each process in the communicator.



MPI_Scatterv

MPI scattering and gathering operations

```
int MPI_Scatterv [ MPI_Iscatterv] (
                [in] const void *data_to_scatter,
                [in] const int *block_sizes,
                [in] const int *block_displacements,
                [in] MPI_Datatype data_type_to_scatter,
                [out] void *receiving_buffer,
                [in] int size_of_receiving_buffer,
                [in] MPI_Datatype data_type_to_receive,
                [in] int rank_that_scatters,
                [in] MPI_Comm comm,
                [out] [ MPI_Request *request ] )
```

- Since it is a global operation, every process in the communicator (comm) must call MPI_Scatterv (or MPI_Iscatterv) with matching parameters (**`data_type`** for send and receive buffers.);
- The information regarding the size of all the partitions and the displacement of each partition must be provided by the root process in the **`block_sizes`** and **`block_displacements`** arrays, respectively. The size of these arrays must be equal to the number of processes in the communicator;
- MPI_Iscatterv as all instantaneous operations, returns immediately and the operation may not have completed when the function returns. The user must ensure that the operation is completed before using the data in the receive buffer. This can be done by calling MPI_Wait or MPI_Test on the request object returned by MPI_Iscatterv.

Example of MPI_Scatterv

MPI scattering and gathering operations

Example of MPI_Scatterv function is shown in the example `8.3.MPI_Scatterv.c`.

```
// Total number of elements in the global vector
int num_global_elements = 80;
// How we want to distribute the elements of the global vector among the processes
int num_local_elements_per_rank[8] = {10, 3, 8, 23, 9, 12, 7, 8};
// Displacements for each process in the global vector
int *displacements = allocate_1d_int(&num_of_ranks);
displacements[0] = 0;
for (int i = 1; i < num_of_ranks; i++)
{
   displacements[i] = displacements[i - 1] + num_local_elements_per_rank[i - 1];
}
// Allocate and initialize the global vector only on the root process
if (mpi_rank == 0)
{
   global_vector = allocate_1d_double(&num_global_elements);
   init_random_vector(10, &num_global_elements, global_vector);
}
// Allocate the local vector for each process
local_vector = allocate_1d_double(&num_local_elements_per_rank[mpi_rank]);
// Scatter operation to distribute parts of the global vector
MPI_Scatterv(&global_vector[0], &num_local_elements_per_rank[0],
             &displacements[0], MPI_DOUBLE,
             &local_vector[0], num_local_elements_per_rank[mpi_rank], MPI_DOUBLE,
             0, MPI_COMM_WORLD);
```

MPI_Gatherv

MPI scattering and gathering operations

```
int MPI_Gatherv [MPI_Igatherv](
                [in] const void *data_to_gather,
                [in] int size_of_data_to_gather,
                [in] MPI_Datatype data_type_to_gather,
                [out] void *receiving_buffer,
                [in] const int *block_sizes,
                [in] const int *block_displacements,
                [in] MPI_Datatype data_type_to_receive,
                [in] int rank_that_gathers,
                [in] MPI_Comm comm,
                [out] [ MPI_Request *request ] )
```

- Since it is a global operation, every process in the communicator (comm) must call MPI_Gatherv (or MPI_Igatherv) with matching parameters (`data_type` for send and receive buffers);
- The information regarding the size of all the partitions and the displacement of each partition must be provided by the root process in the `block_sizes` and `block_displacements` arrays, respectively. The size of these arrays must be equal to the number of processes in the communicator;
- MPI_Igatherv as all instantaneous operations, returns immediately and the operation may not have completed when the function returns. The user must ensure that the operation is completed before using the data in the receive buffer. This can be done by calling MPI_Wait or MPI_Test on the request object returned by MPI_Igatherv.

Example of MPI_Gatherv

MPI scattering and gathering operations

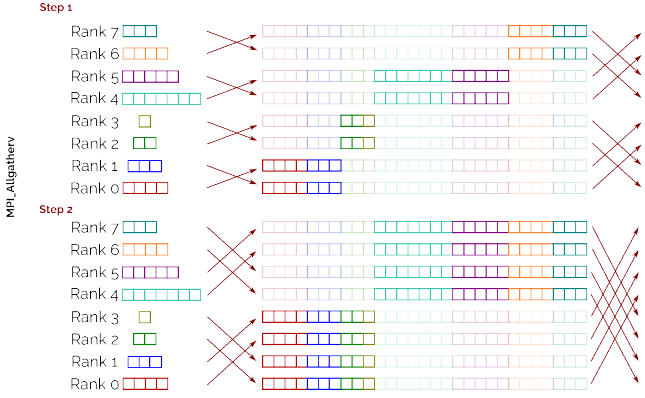
Example of MPI_Gatherv function is shown in the example `8.4.MPI_Gatherv.c`.

```
// Total number of elements in the global vector
int num_global_elements = 80;
// How we want to distribute the elements of the global vector among the processes
int num_local_elements_per_rank[8] = {10, 3, 8, 23, 9, 12, 7, 8};
// Displacements for each process in the global vector
int *displacements = allocate_1d_int(&num_of_ranks);
displacements[0] = 0;
for (int i = 1; i < num_of_ranks; i++)
{
   displacements[i] = displacements[i - 1] + num_local_elements_per_rank[i - 1];
}
// Allocate and initialize the local vector for each process
local_vector = allocate_1d_double(&num_local_elements_per_rank[mpi_rank]);
init_random_vector(mpi_rank, &num_local_elements_per_rank[mpi_rank], local_vector);
// Allocate the global vector only on the root process
if (mpi_rank == 0)
{
   global_vector = allocate_1d_double(&num_global_elements);
}
// Gatherv process to collect local vectors into the global vector
MPI_Gatherv(&local_vector[0], num_local_elements_per_rank[mpi_rank], MPI_DOUBLE,
         &global_vector[0], &num_local_elements_per_rank[0],
         &displacements[0],
         MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

MPI_Allgather and MPI_Allgatherv

MPI scattering and gathering operations

If we want to gather data from all processes in the communicator and distribute it to all processes, we can use the MPI_AllGather and MPI_AllGatherv functions.



MPI_Allgather and MPI_Allgatherv

MPI scattering and gathering operations

The MPI_Allgather and MPI_Allgatherv functions are similar to MPI_Gather and MPI_Gatherv, but it gathers data from all processes and distributes the gathered data to all processes in the communicator.

```
int MPI_Allgather [MPI_Iallgather](
                [in] const void *data_to_gather,
                [in] int size_of_data_to_gather,
                [in] MPI_Datatype data_type_to_gather,
                [out] void *receiving_buffer,
                [in] MPI_Datatype data_type_to_receive,
                [in] MPI_Comm comm,
                [out] [ MPI_Request *request ]
                )
```

```
int MPI_Allgatherv [MPI_Iallgatherv](
                [in] const void *data_to_gather,
                [in] int size_of_data_to_gather,
                [in] MPI_Datatype data_type_to_gather,
                [out] void *receiving_buffer,
                [in] const int *block_sizes,
                [in] const int *block_displacements,
                [in] MPI_Datatype data_type_to_receive,
                [in] MPI_Comm comm,
                [out] [ MPI_Request *request ]
                )
```

In both functions we do not need to specify the rank of the root process, since all processes are involved in the operation.

Example of MPI_Allgatherv

MPI scattering and gathering operations

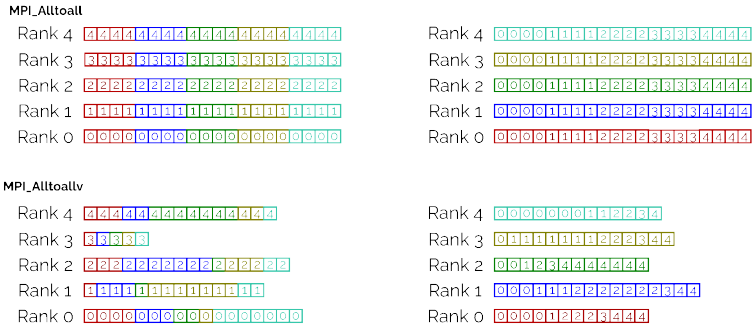
Example of MPI_Allgatherv function is shown in the example `8.5.MPI_Allgatherv.c`.

```
// Total number of elements in the global vector
int num_global_elements = 80;
// How the elements are distributed within the processes
int num_local_elements_per_rank[8] = {10, 3, 8, 23, 9, 12, 7, 8};
// Displacements for each process in the global vector
int *displacements = (int *)malloc(num_of_ranks * sizeof(int));
displacements[0] = 0;
for (int i = 1; i < num_of_ranks; i++)
{
   displacements[i] = displacements[i - 1] + num_local_elements_per_rank[i - 1];
}
// Initialize the local vector for each process
local_vector = allocate_1d_double(&num_local_elements_per_rank[mpi_rank]);
init_random_vector(mpi_rank, &num_local_elements_per_rank[mpi_rank], local_vector);
// Allocate the global vector to gather all local vectors
global_vector = allocate_1d_double(&num_global_elements);
// Allgatherv operation to gather local vectors into the global vector
MPI_Allgatherv(&local_vector[0], num_local_elements_per_rank[mpi_rank], MPI_DOUBLE,
               &global_vector[0], &num_local_elements_per_rank[0],
               &displacements[0], MPI_DOUBLE, MPI_COMM_WORLD);
```

MPI_Alltoall and MPI_Alltoallv

MPI scattering and gathering operations

MPI_Alltoall is an extension of MPI_Allgather to the case where each process sends distinct data to each of the receivers.



The *j*th block sent from process *i* is received by process *j* and is placed in the *i*th block of the receiving buffer.

MPI_Alltoallv

MPI scattering and gathering operations

The input and output variables are similar to those defined for MPI_Allgather and MPI_Allgatherv, with the difference that each process can send and receive different amounts of data to/from each process in the communicator.

```
int MPI_Alltoall [MPI_IAlltoall](
                [in] const void *data_to_send,
                [in] int size_of_data_to_send,
                [in] MPI_Datatype data_type_to_send,
                [out] void *receiving_buffer,
                [in] int size_of_data_to_reveive,
                [in] MPI_Datatype data_type_to_receive,
                [in] MPI_Comm comm,
                [out] [ MPI_Request *request ] )
```

```
int MPI_Alltoallv [MPI_IAlltoallv](
                [in] const void *data_to_send,
                [in] int size_of_data_to_send,
                [in] MPI_Datatype data_type_to_send,
                [out] void *receiving_buffer,
                [in] const int *block_sizes,
                [in] const int *block_displacements,
                [in] MPI_Datatype data_type_to_receive,
                [in] MPI_Comm comm,
                [out] [ MPI_Request *request ] )
```

Example of MPI_Alltoallv

MPI scattering and gathering operations

Example of MPI_Alltoallv function is shown in the example `8.6.MPI_Alltoallv.c`.

All ranks initialize a local vector with random blocks of different sizes and their displacements.

```
int *size_send_blocks = allocate_1d_int(&num_of_ranks);
int *send_displacements = allocate_1d_int(&num_of_ranks);
```

The function MPI_Alltoall is called to exchange the sizes of the blocks to be sent and received among all processes in the communicator.

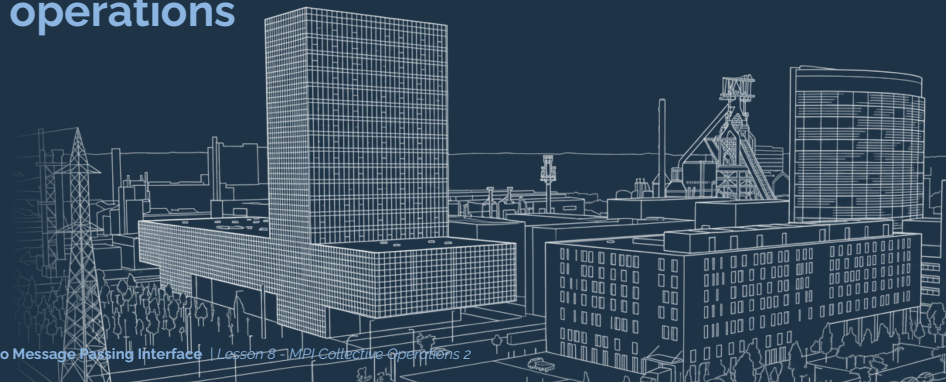
```
MPI_Alltoall(size_send_blocks, 1, MPI_INT,
             size_receive_blocks, 1, MPI_INT, MPI_COMM_WORLD);
```

Afterwards, the displacements for the received blocks are calculated based on the sizes of the received blocks.
Finally, the MPI_Alltoallv function is called to exchange the actual data blocks among all processes in the communicator.

```
MPI_Alltoallv(&send_buffer[0], &size_send_blocks[0], &send_displacements[0], MPI_DOUBLE,
   &receive_buffer[0], &size_receive_blocks[0], &receive_displacements[0], MPI_DOUBLE,
   MPI_COMM_WORLD);
```

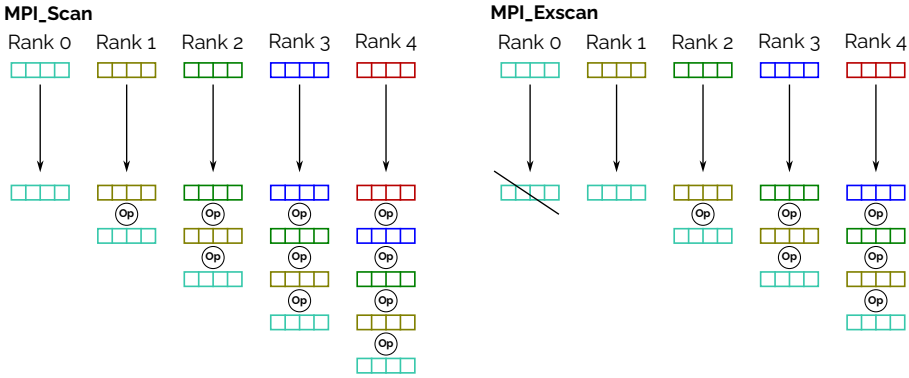
Université du Luxembourg

MPI scan operations



MPI_Scan and MPI_Exscan functions

MPI scan operations



MPI_Scan and MPI_Exscan functions

MPI scan operations

```
int MPI_Scan [ MPI_Iscan ] (
            [in] const void *send_buffer,
            [out] const void *receive_buffer,
            [in] int size_of_data,
            [in] MPI_Datatype data_type,
            [in] MPI_Op reduction_operation,
            [in] MPI_Comm comm,
            [out] [MPI_Request *request ]
            )
```

each process receives the result of the reduction operation applied to all processes with a lower rank, including itself.

```
int MPI_Exscan [ MPI_Iexscan ] (
            [in] const void *send_buffer,
            [out] const void *receive_buffer,
            [in] int size_of_data,
            [in] MPI_Datatype data_type,
            [in] MPI_Op reduction_operation,
            [in] MPI_Comm comm,
            [out] [MPI_Request *request ]
            )
```

each process receives the result of the reduction operation applied to all processes with a lower rank, excluding itself.

MPI_Scan and MPI_Exscan function

MPI scan operations

Example of MPI_Scan and MPI_Exscan function is shown in the example `8.4.MPI_Scan.c` and `8.5.MPI_Exscan.c`.

The simple use of MPI_Scan and MPI_Exscan functions is shown in the following code snippets.

```
int local_val = mpi_rank;
int scan_result = 0;

MPI_Scan(&local_val, &scan_result, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

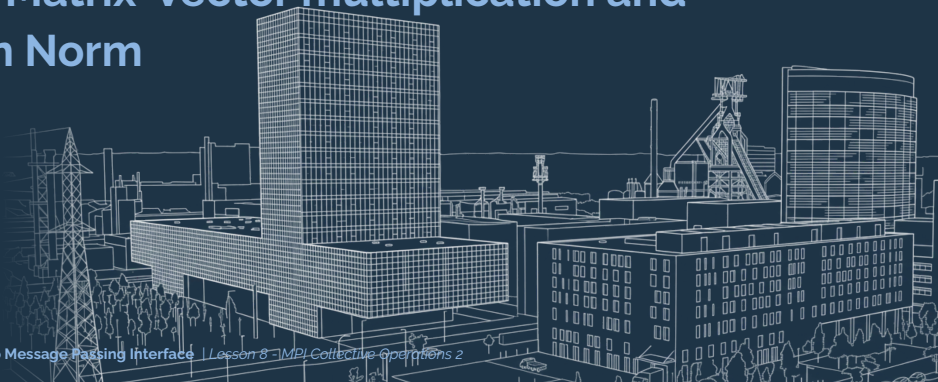
```
int local_val = mpi_rank;
int scan_result = 0;

MPI_Exscan(&local_val, &scan_result, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```

Any reduce operation can be used in the MPI_Scan and MPI_Exscan functions, such as MPI_SUM, MPI_PROD, MPI_MAX, MPI_MIN ...

Université du Luxembourg

Exercise: Matrix-Vector multiplication and Euclidean Norm



Definition of matrix-vector multiplication and the norm of a vector

Exercise: Matrix-Vector multiplication and Euclidean Norm

We consider a matrix $\mathbf{A}$ with *N* rows and *M* columns, and a vector $\mathbf{b}$ of length *M*, both containing double precision elements.

We define the **matrix-vector product** between the transposed of $\mathbf{A}$ and the vector $\mathbf{b}$ as $\mathbf{c} = \mathbf{A}\mathbf{b}$ and the single element of the $\mathbf{c}$ will be given by

$$\mathbf{c}_i = \sum_{j=1}^{N} \mathbf{A}_{ij}\mathbf{b}_j.$$

We define the **dot product** between two vectors as

$$d = \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{N} a_i b_i, \tag{1}$$

and **the norm** $\mathcal{N}$ of a vector as:

$$\mathcal{N} = \sqrt{\mathbf{a} \cdot \mathbf{a}} = \sqrt{\sum_{i=1}^{N} a_i^2}. \tag{2}$$

Exercise: Compute the norm of a vector $\mathbf{c}$ result of $\mathbf{c} = \mathbf{Ab}$

Exercise: Matrix-Vector multiplication and Euclidean Norm

Exercise: Write an MPI code to parallelize the computation of the norm $\mathcal{N}$ of a vector *c*, result of a Matrix-Vector operation do a Matrix vector operation $\mathbf{c} = \mathbf{Ab}$.

The code will use any type of functions that you prefer, in particular `MPI_*Scatter/MPI_*Gather`, `MPI_*Bcast` and `MPI_*Reduce`.

The parallel execution of the mathematical operations must also consider the case in which the number of rows cannot be split evenly between the cores (`MPI_*Scatterv/MPI_*Gatherv`).

1. Rank 0 Initializes matrix $\mathbf{A}$ (5000×200 elements)and vector $\mathbf{b}$ (100 elements) with random double precision numbers distributed in the interval [-1:1] using the functions in `array.h`. The initial random seed to be used is 10.

2. Rank 0 communicates to all other ranks the vector $\mathbf{b}$ and parts of the matrix $\mathbf{A}$.

3. Each process executes the local matrix-vector product using the functions in `linearalgebra.h`, obtaining parts of the of the vector $\mathbf{c}$.

Exercise: Compute the norm of a vector $\mathbf{c}$ result of $\mathbf{c} = \mathbf{Ab}$

Exercise: Matrix-Vector multiplication and Euclidean Norm

4. Each process computes parts of the dot product of the vector $\mathbf{c}$ with itself (parts of the $\sum_{i=1}^{N} c_i^2$ sum) necessary to compute the norm (`ddot` in `linearalgebra.h`).

5. The results of the partial dot products obtained for each process are summed through a reduce operation on rank 0.

6. Rank 0 computes the square root of the dot product $\sqrt{\sum_{i=1}^{N} c_i^2}$, equal to the norm of the vector $\mathbf{c}$

7. Rank 0 prints the value of the norm.

Measure the times for "Initialization", "MPI communications", "Matrix operations", "Reduce Operations" with the MPI function `MPI_Wtime()`.

Q1. How does the efficiency of Scatter and Scatterv compare?

Q2. How do the communication operations scale as a function of the number of processors?